

gRPC & Protocol Buffers 勉強会

Go言語による実践ハンズオン

アジェンダ

1. **Protocol Buffers とは** — 概要・特徴・なぜ使うのか
2. **RPC / gRPC とは** — RPC の概念・gRPC の特徴・REST との違い
3. **HTTP/2 と Cookie** — HTTP/1.1 の課題・HTTP/2・gRPC のセッション管理
4. **環境構築** — 必要なツールのインストール
5. **ハンズオン Part 1: Protocol Buffers** — `.proto` 定義 → コード生成 → シリアライズ
6. **ハンズオン Part 2: gRPC** — サービス定義 → サーバー/クライアント → 4つの通信パターン
7. **発展: Interceptor・認証・TLS**

Protocol Buffers とは

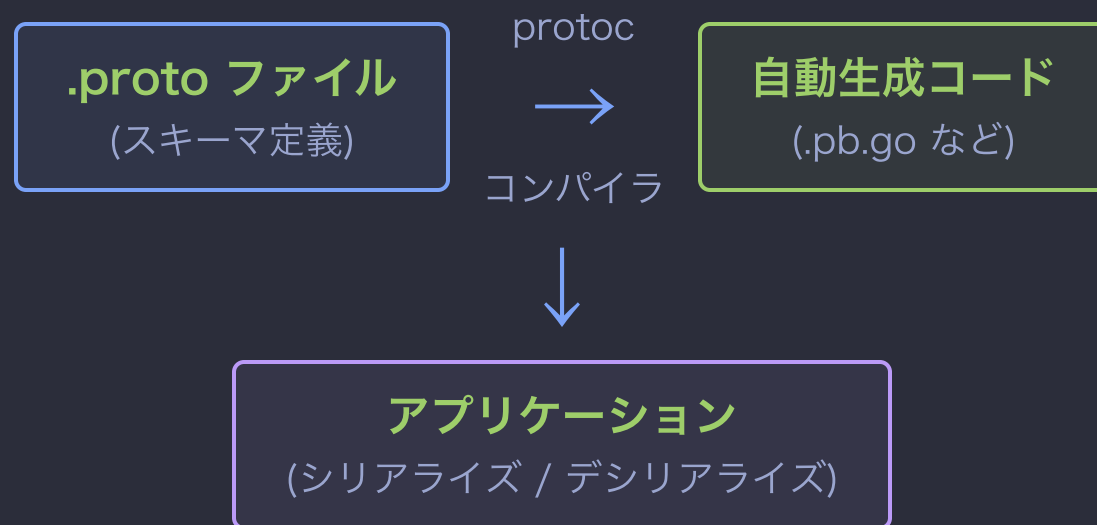
概要

- Google が開発した言語・プラットフォーム中立なシリアライゼーションフォーマット
- `.proto` ファイルでデータ構造を定義し、各言語のコードを自動生成
- JSON/XML に比べて高速・軽量・型安全

なぜ Protocol Buffers を使うのか？

比較項目	JSON	Protocol Buffers
フォーマット	テキスト (人間が読める)	バイナリ (コンパクト)
スキーマ	なし (自由記述)	<code>.proto</code> で厳密に定義
型安全性	弱い	強い
サイズ	大きい	小さい
パース速度	遅い	速い
コード生成	なし	自動生成

Protocol Buffers の仕組み



ポイント

- **.proto** ファイルが**信頼できる唯一の情報源 (Single Source of Truth)**
- Go, Java, Python, C++ など**多言語対応**
- **フィールドはタグ番号 (1, 2, ...)** で識別される — シリアライズデータにフィールド名は含まれない (JSON との大きな違い)

RPC / gRPC とは

RPC (Remote Procedure Call)

- ネットワーク越しに、あたかも**ローカル関数を呼ぶように**別サーバーの処理を実行する仕組み
- REST: 「リソース (URL)」を操作する思想 → `GET /files`, `POST /files`
- RPC: 「関数 (手続き)」を呼び出す思想 → `ListFiles()`, `Upload()`

```
// クライアント側のコード — ローカル関数のように見えるが、実際はサーバー側で実行される
res, err := client.ListFiles(ctx, &pb.ListFilesRequest{})
```

gRPC

- Google が開発した**高性能なオープンソース RPC フレームワーク** (g = gRPC 自身の再帰略語)
- **HTTP/2 + Protocol Buffers** を組み合わせ、高速・型安全な通信を実現
- `.proto` ファイルからサーバー/クライアント双方のコードを自動生成
- マイクロサービス間通信で広く採用 (Google, Netflix, Square 等)

gRPC の主な特徴:

- バイナリ通信で JSON の **3~10倍高速** / **4つの通信パターン**対応
- **多言語対応** — Go, Java, Python, C++, Rust など 10+ 言語の公式サポート

REST API との違い - HTTP/2

REST vs gRPC

比較項目	REST API	gRPC
プロトコル	HTTP/1.1	HTTP/2
データ形式	JSON (テキスト)	Protocol Buffers (バイナリ)
API定義	OpenAPI/Swagger (任意)	<code>.proto</code> ファイル (必須)
ストリーミング	限定的	双方向ストリーミング対応
コード生成	ツール依存	公式サポート
ブラウザ対応	ネイティブ	gRPC-Web※ が必要

※ **gRPC-Web**: ブラウザから gRPC サーバーを呼び出すプロトコル。HTTP/1.1 上でも動作し、Envoy 等のプロキシを介して変換。Unary と Server Streaming のみ対応。

HTTP/1.1 の課題 → HTTP/2 で解決

HTTP/1.1 の課題	HTTP/2 の解決策
Head-of-Line Blocking — 1接続1リクエストずつ	多重化 — 1接続で複数リクエストを並行処理
ヘッダー肥大化 — Cookie等を毎回そのまま送信	HPACK圧縮 — 差分のみ送信、Cookie もほぼゼロコストに
テキストベース — パースが遅い	バイナリフレーミング — パース高速化
クライアント起点のみ	サーバープッシュ — サーバーから能動的に送信可能

gRPC と Cookie / セッション管理

gRPC は Cookie を使わない

- HTTP/2 上で動作するが、**Cookie ヘッダーは使用しない**
- 代わりに **メタデータ (metadata)** で認証情報を送る

```
// gRPC の認証: メタデータに Bearer トークンを付与
md := metadata.New(map[string]string{"authorization": "Bearer <token>"})
ctx := metadata.NewOutgoingContext(context.Background(), md)
```

REST (Cookie) vs gRPC (メタデータ)

	REST	gRPC
認証情報の送信	Cookie (自動付与)	メタデータ (明示的に付与)
ステート管理	サーバー側セッション	ステートレス (トークンベース)
CSRF リスク	あり (Cookie 自動送信)	なし (明示的に付与するため)

自動/明示の根拠: Cookie は **ブラウザ仕様 (RFC 6265)** によりブラウザが同ドメインへのリクエストに自動付与する。一方 gRPC メタデータには自動付与の仕組みがなく、クライアントが `metadata.NewOutgoingContext()` 等で**毎回明示的に付与**する必要がある。

補足: gRPC-Web 経由でブラウザから通信する場合は HTTP リクエストとなるため、Cookie の送受信が可能。既存の Cookie ベース認証基盤と組み合わせる場合に利用される。

gRPC の 4 つの通信パターン

パターン	通信の流れ		ユースケース
1. Unary 1 req → 1 res	Client	req → res ← Server	通常のAPI呼び出し
2. Server Streaming 1 req → N res	Client	req → res ← res ← res ← Server	ファイルDL
3. Client Streaming N req → 1 res	Client	req → req → req → res ← Server	ファイルUL
4. Bidirectional N req ↔ N res	Client	req → res ← req → res ← Server	チャット等 リアルタイム通信

環境構築

1. Go のインストール

```
# macOS (Homebrew)
brew install go

# バージョン確認
go version
# go version go1.24.x darwin/arm64
```

2. Protocol Buffers コンパイラ (protoc)

```
# macOS (Homebrew)
brew install protobuf

# バージョン確認
protoc --version
# libprotoc 2x.x
```

3. Go 用プラグインのインストール

```
# protoc-gen-go: .proto → Go の構造体コードを生成
brew install protoc-gen-go

# バージョン確認
protoc-gen-go --version
# protoc-gen-go v1.36.11

# protoc-gen-go-grpc: .proto → gRPC サービスコードを生成
brew install protoc-gen-go-grpc

# バージョン確認
protoc-gen-go-grpc --version
# protoc-gen-go-grpc 1.6.1
```

ハンズオン Part 1 — Protocol Buffers 基礎編

プロジェクト構成 (protobuf-lesson)

```
protobuf-lesson/
├── proto/
│   ├── employee.proto  ← スキーマ定義
│   └── date.proto      ← インポート用スキーマ
├── pb/
│   ├── employee.pb.go  ← 自動生成コード
│   └── date.pb.go      ← 自動生成コード
├── main.go             ← アプリケーションコード
├── go.mod
└── go.sum
```

.proto ファイルの書き方 - 基本構文

```
syntax = "proto3"; // proto3 を指定（省略すると proto2 になる）
```

```
package date;           // パッケージ名  
option go_package = ".pb"; // Go のパッケージパス
```

```
message Date {  
    int32 year = 1; // タグ番号 = 1  
    int32 month = 2; // タグ番号 = 2  
    int32 day = 3; // タグ番号 = 3  
}
```

タグ番号のルール

- フィールドは**タグ番号**で識別される（フィールド名ではない）
- **1 ~ 15**: 1バイトで表現 → **よく使うフィールドに割り当てる**
- **16 ~ $2^{29}-1$** : 2バイト以上 → あまり使わないフィールド向け
- **19000 ~ 19999**: protobuf の予約領域 → **使用不可**
- 連番である必要はないが、**一意でなければならない**

.proto ファイルの書き方 - 高度な型

```
message Employee {
  int32 id = 1;
  string name = 2;
  string email = 3;
  Occupation occupation = 4;          // enum 型
  repeated string phone_number = 5; // 配列 (リスト)
  map<string, Company.Project> project = 6; // マップ
  oneof profile {                     // いずれか1つの値を持つ
    string text = 7;
    Video video = 8;
  }
  date.Date birthday = 9;             // 別パッケージの型を参照
}

enum Occupation {
  OCCUPATION_UNKNOWN = 0; // enum の最初の値は必ず 0
  ENGINEER = 1;
  DESIGNER = 2;
  MANAGER = 3;
}
```

Protocol Buffers のデフォルト値

フィールドが設定されていない場合、型に応じたデフォルト値が使われる:

型	デフォルト値
<code>string</code>	<code>""</code> (空文字列)
<code>int32</code> / <code>int64</code>	<code>0</code>
<code>float</code> / <code>double</code>	<code>0.0</code>
<code>bool</code>	<code>false</code>
<code>enum</code>	<code>0</code> (最初の値)
<code>repeated</code>	<code>[]</code> (空リスト)
<code>map</code>	<code>{}</code> (空マップ)
<code>oneof</code>	<code>nil</code> (未設定)
<code>message</code>	<code>nil</code>

注意: proto3 ではデフォルト値のフィールドはシリアライズ時に省略される

コード生成と生成されるコード

protoc コマンドの実行

```
cd protobuf-lesson
protoc --go_out=. proto/*.proto # .proto → Go コードを生成
```

生成ファイル: `pb/date.pb.go` (`Date` 構造体) / `pb/employee.pb.go` (`Employee`, `Occupation` 等)

生成されるコードの中身

```
// pb/employee.pb.go (自動生成 - 編集しないこと)
type Employee struct {
    Id          int32          `protobuf:"varint,1,..."`
    Name        string         `protobuf:"bytes,2,..."`
    Email       string         `protobuf:"bytes,3,..."`
    Occupation  Occupation    `protobuf:"varint,4,..."`
    PhoneNumber []string       `protobuf:"bytes,5,..."`
    Project     map[string]*Company_Project `protobuf:"bytes,6,..."`
    // ...
}
```

- `.proto` のフィールド定義が Go の構造体フィールドに変換される
- タグ番号は構造体タグとして保持 / `repeated` → スライス、`map` → Go の `map`

Go で Protocol Buffers を使う

メッセージの構築

```
employee := &pb.Employee{
    Id: 1, Name: "Suzuki", Email: "test@example.com",
    Occupation: pb.Occupation_ENGINEER,
    PhoneNumber: []string{"080-1234-5678", "090-1234-5678"},
    Project:     map[string]*pb.Company_Project{"ProjectX": {}},
    Profile:     &pb.Employee_Text{Text: "My name is Suzuki"},
    Birthday:    &pb.Date{Year: 2000, Month: 1, Day: 1},
}
```

シリアライズ / デシリアライズ

JSON — デバッグ・外部API連携向け

```
// Protobuf → JSON
jsonBytes, _ := protojson.Marshal(employee)

// JSON → Protobuf
read := &pb.Employee{}
protojson.Unmarshal(jsonBytes, read)
```

バイナリ — サービス間通信・高速・コンパクト

```
// Protobuf → バイナリ
binData, _ := proto.Marshal(employee)
os.WriteFile("test.bin", binData, 0644)

// バイナリ → Protobuf
in, _ := os.ReadFile("test.bin")
read := &pb.Employee{}
proto.Unmarshal(in, read)
```

ハンズオン Part 2 — gRPC 実践編

プロジェクト構成 (grpc-lesson)

```
grpc-lesson/
├── proto/
│   └── file.proto      ← サービス + メッセージ定義
├── pb/
│   ├── file.pb.go      ← メッセージ型 (自動生成)
│   └── file_grpc.pb.go ← gRPC サービス (自動生成)
├── server/
│   ├── main.go         ← サーバー実装
│   └── ssl/            ← サーバー証明書 (mkcert で生成)
│       ├── localhost.pem
│       └── localhost-key.pem
├── client/
│   ├── main.go         ← クライアント実装
│   └── ssl/            ← クライアント検証用ルート CA
│       └── rootCA.pem
├── storage/            ← テスト用データ
│   ├── name.txt
│   └── sports.txt
├── go.mod
└── go.sum
```


gRPC のサービス定義 (.proto)

メッセージ定義

```
syntax = "proto3";
package file;
option go_package = "./pb";

message ListFilesRequest {};
message ListFilesResponse { repeated string filenames = 1; };
message DownloadRequest { string filename = 1; };
message DownloadResponse { bytes data = 1; };
message UploadRequest { bytes data = 1; };
message UploadResponse { int32 size = 1; };
message UploadAndNotifyProgressRequest { bytes data = 1; };
message UploadAndNotifyProgressResponse { string msg = 1; };
```

サービス定義 — **stream** キーワードがストリーミングを表す

```
service FileService {
  rpc ListFiles(ListFilesRequest) returns (ListFilesResponse);           // Unary
  rpc Download(DownloadRequest) returns (stream DownloadResponse);        // Server Streaming
  rpc Upload(stream UploadRequest) returns (UploadResponse);              // Client Streaming
  rpc UploadAndNotifyProgress(stream UploadAndNotifyProgressRequest)      // Bidirectional
    returns (stream UploadAndNotifyProgressResponse);
}
```

コード生成 (gRPC) と生成されるインターフェース

protoc コマンド

```
cd grpc-lesson
protoc --go_out=. --go-grpc_out=. proto/file.proto # メッセージ型 + gRPC サービスコードを同時に生成
```

生成ファイル: `pb/file.pb.go` (メッセージ型) / `pb/file_grpc.pb.go` (サービスIF + クライアントスタブ)

file_grpc.pb.go に生成される主要なインターフェース

```
// サーバー側: このインターフェースを実装する
type FileServiceServer interface {
    ListFiles(context.Context, *ListFilesRequest) (*ListFilesResponse, error)
    Download(*DownloadRequest, FileService_DownloadServer) error
    Upload(FileService_UploadServer) error
    UploadAndNotifyProgress(FileService_UploadAndNotifyProgressServer) error
}

// クライアント側: このクライアントを使って呼び出す
type FileServiceClient interface {
    ListFiles(ctx context.Context, in *ListFilesRequest, ...) (*ListFilesResponse, error)
    Download(ctx context.Context, in *DownloadRequest, ...) (FileService_DownloadClient, error)
    Upload(ctx context.Context, ...) (FileService_UploadClient, error)
    UploadAndNotifyProgress(ctx context.Context, ...) (FileService_UploadAndNotifyProgressClient, error)
}
```

サーバー実装 - 基本構造

```
package main

import (
    "fmt"
    "log"
    "net"
    "grpc-lesson/pb"
    "google.golang.org/grpc"
)

// サーバー構造体 (自動生成の UnimplementedXxxServer を埋め込む)
type server struct {
    pb.UnimplementedFileServiceServer
}

func main() {
    lis, err := net.Listen("tcp", "localhost:50051")
    if err != nil {
        log.Fatalf("Failed to listen: %v", err)
    }

    s := grpc.NewServer()
    pb.RegisterFileServiceServer(s, &server{})

    fmt.Println("Server is running on port 50051")
    if err := s.Serve(lis); err != nil {
        log.Fatalf("Failed to serve: %v", err)
    }
}
```

パターン 1: Unary RPC (ListFiles)

サーバー側

```
func (*server) ListFiles(  
    ctx context.Context,  
    req *pb.ListFilesRequest,  
) (*pb.ListFilesResponse, error) {  
    fmt.Println("=====[Unary] ListFiles invoked =====")  
  
    paths, err := os.ReadDir("../storage")  
    if err != nil {  
        return nil, err  
    }  
  
    var filenames []string  
    for _, path := range paths {  
        if !path.IsDir() {  
            filenames = append(filenames, path.Name())  
        }  
    }  
  
    fmt.Printf(" → returning %d file(s): %v\n", len(filenames), filenames)  
    return &pb.ListFilesResponse{Filenames: filenames}, nil  
}
```

ポイント: 通常の関数呼び出しと同じ感覚。リクエストを受け取り、レスポンスを返す。

パターン 1: Unary RPC (ListFiles) - クライアント側

```
func main() {
    // TLS ルート証明書を読み込み
    creds, err := credentials.NewClientTLSFromFile("ssl/rootCA.pem", "")
    if err != nil {
        log.Fatalf("Failed to load TLS credentials: %v", err)
    }

    // gRPC 接続を確立
    conn, err := grpc.NewClient(
        "localhost:50051",
        grpc.WithTransportCredentials(creds),
    )
    if err != nil {
        log.Fatalf("Failed to connect: %v", err)
    }
    defer conn.Close()

    // クライアントを生成
    client := pb.NewFileServiceClient(conn)
    callListFiles(client)
}

func callListFiles(client pb.FileServiceClient) {
    fmt.Println("===== [Unary] ListFiles =====")
    // 認証トークンをメタデータに付与
    md := metadata.New(map[string]string{"authorization": "Bearer test-token"})
    ctx := metadata.NewOutgoingContext(context.Background(), md)

    res, err := client.ListFiles(ctx, &pb.ListFilesRequest{})
    if err != nil {
        log.Fatalf("Failed to call ListFiles: %v", err)
    }

    filenames := res.GetFilenames()
    fmt.Printf("Found %d file(s):\n", len(filenames))
    for i, name := range filenames {
        fmt.Printf("  [%d] %s\n", i+1, name)
    }
}
```

パターン 2: Server Streaming (Download) - サーバー側

ファイルを5バイトずつストリーミング送信:

```
func (*server) Download(
    req *pb.DownloadRequest,
    stream pb.FileService_DownloadServer,
) error {
    fmt.Println("===== [Server Streaming] Download invoked =====")
    filename := req.GetFilename()
    path := "../storage/" + filename

    if _, err := os.Stat(path); os.IsNotExist(err) {
        return status.Errorf(codes.NotFound, "file not found: %s", filename)
    }

    file, err := os.Open(path)
    if err != nil { return err }
    defer file.Close()

    buf := make([]byte, 5) // 5バイトずつ読み取り
    chunkIdx := 0
    for {
        n, err := file.Read(buf)
        if n == 0 || err == io.EOF { break }
        if err != nil { return err }

        stream.Send(&pb.DownloadResponse{Data: buf[:n]}) // 複数回送信
        chunkIdx++
        fmt.Printf(" → sent chunk #%d (%d bytes): %q\n", chunkIdx, n, string(buf[:n]))
        time.Sleep(1 * time.Second)
    }
    return nil
}
```

パターン 2: Server Streaming (Download) - クライアント側

```
func callDownload(client pb.FileServiceClient) {
    fmt.Println("===== [Server Streaming] Download =====")
    // タイムアウト付きコンテキスト
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()

    req := &pb.DownloadRequest{Filename: "name.txt"}
    stream, err := client.Download(ctx, req)
    if err != nil {
        log.Fatalf("Failed to call Download: %v", err)
    }

    chunkIdx := 0
    for {
        res, err := stream.Recv() // 複数回受信
        if err == io.EOF { break } // ストリーム終了
        if err != nil {
            resErr, ok := status.FromError(err)
            if ok {
                if resErr.Code() == codes.NotFound {
                    log.Fatalf("Error: %v", resErr.Message())
                } else if resErr.Code() == codes.DeadlineExceeded {
                    log.Fatalln("deadline exceeded")
                }
            }
        }
        chunkIdx++
        data := res.GetData()
        fmt.Printf("  chunk #%d (%d bytes): %q\n", chunkIdx, len(data), string(data))
    }
}
```

パターン 3: Client Streaming (Upload) - サーバー側

クライアントからのストリームを受信:

```
func (*server) Upload(stream pb.FileService_UploadServer) error {
    fmt.Println("===== [Client Streaming] Upload invoked =====")

    var buf bytes.Buffer
    chunkIdx := 0
    for {
        req, err := stream.Recv() // クライアントからの送信を繰り返し受信
        if err == io.EOF {
            // クライアントが送信完了 → レスポンスを返して終了
            res := &pb.UploadResponse{Size: int32(buf.Len())}
            return stream.SendAndClose(res)
        }
        if err != nil { return err }

        data := req.GetData()
        chunkIdx++
        fmt.Printf(" ← received chunk #%d (%d bytes): %q\n", chunkIdx, len(data), string(data))
        buf.Write(data)
    }
}
```

ポイント: `stream.Recv()` でループし、`io.EOF` で終了を検知。

`stream.SendAndClose()` で最終レスポンスを返す。

パターン 3: Client Streaming (Upload) - クライアント側

```
func callUpload(client pb.FileServiceClient) {
    fmt.Println("===== [Client Streaming] Upload =====")
    file, err := os.Open("../storage/sports.txt")
    if err != nil { log.Fatalf("Failed to open file: %v", err) }
    defer file.Close()

    stream, err := client.Upload(context.Background())
    if err != nil { log.Fatalf("Failed to call Upload: %v", err) }

    buf := make([]byte, 5)
    chunkIdx := 0
    for {
        n, err := file.Read(buf)
        if n == 0 || err == io.EOF { break }
        if err != nil { log.Fatalf("Failed to read: %v", err) }

        req := &pb.UploadRequest{Data: buf[:n]}
        stream.Send(req) // 複数回送信
        chunkIdx++
        fmt.Printf(" → sent chunk #%d (%d bytes): %q\n", chunkIdx, n, string(buf[:n]))
        time.Sleep(1 * time.Second)
    }

    // 送信完了 & レスポンス受信
    res, err := stream.CloseAndRecv()
    if err != nil { log.Fatalf("Failed: %v", err) }
    fmt.Printf("Server reported size: %d bytes\n", res.GetSize())
}
```

パターン 4: Bidirectional Streaming - サーバー側

受信のたびに進捗を返す:

```
func (*server) UploadAndNotifyProgress(
    stream pb.FileService_UploadAndNotifyProgressServer,
) error {
    fmt.Println("===== [Bidirectional Streaming] UploadAndNotifyProgress invoked =====")

    size := 0
    chunkIdx := 0
    for {
        req, err := stream.Recv()
        if err == io.EOF { return nil }
        if err != nil { return err }

        data := req.GetData()
        chunkIdx++
        size += len(data)
        fmt.Printf(" ← received chunk #%d (%d bytes): %q\n", chunkIdx, len(data), string(data))

        // 受信するたびにレスポンスを送信
        msg := fmt.Sprintf("Received %d bytes", size)
        stream.Send(&pb.UploadAndNotifyProgressResponse{Msg: msg})
        fmt.Printf(" → [progress] %s\n", msg)
    }
}
```

ポイント: `Recv()` と `Send()` を同じストリームで使用

パターン 4: Bidirectional Streaming - クライアント側 (送信)

```
func callUploadAndNotifyProgress(client pb.FileServiceClient) {
    fmt.Println("===== [Bidirectional Streaming] UploadAndNotifyProgress =====")
    stream, _ := client.UploadAndNotifyProgress(context.Background())

    // 送信用 goroutine
    go func() {
        file, _ := os.Open("../storage/sports.txt")
        defer file.Close()
        buf := make([]byte, 5)
        chunkIdx := 0
        for {
            n, err := file.Read(buf)
            if n == 0 || err == io.EOF { break }
            stream.Send(&pb.UploadAndNotifyProgressRequest{Data: buf[:n]})
            chunkIdx++
            fmt.Printf(" → [send]      chunk #%d (%d bytes): %q\n", chunkIdx, n, string(buf[:n]))
            time.Sleep(1 * time.Second)
        }
        stream.CloseSend() // 送信完了を通知
    }()
}
```

ポイント: goroutine で送信と受信を並行処理する

パターン 4: Bidirectional Streaming - クライアント側 (受信)

```
// 受信用 goroutine
ch := make(chan struct{})
go func() {
    for {
        res, err := stream.Recv()
        if err == io.EOF { break }
        if err != nil {
            log.Fatalf("Failed to receive data: %v", err)
        }
        fmt.Printf(" ← [progress] %s\n", res.GetMsg())
    }
    close(ch)
}()

fmt.Println("Waiting for progress notifications...")
<-ch // 受信完了を待機
fmt.Println("UploadAndNotifyProgress finished")
}
```

ポイント: チャンネル `ch` で受信 goroutine の完了を待機

発展: Interceptor (ミドルウェア)

ログ出力 Interceptor

```
func myLogging() grpc.UnaryServerInterceptor {
    return func(
        ctx context.Context,
        req interface{},
        info *grpc.UnaryServerInfo,
        handler grpc.UnaryHandler,
    )(interface{}, error) {
        log.Printf("request data: %+v", req)    // リクエストをログ出力

        resp, err := handler(ctx, req)         // 本来の実行
        if err != nil { return nil, err }

        log.Printf("response data: %+v", resp) // レスポンスをログ出力
        return resp, nil
    }
}
```

Interceptor とは?: HTTP ミドルウェアの gRPC 版。

リクエストの前後に共通処理（ログ、認証、メトリクス等）を挿入できる。

発展: 認証 (Authentication)

gRPC のメタデータは HTTP ヘッダーに相当。 `metadata.NewOutgoingContext()` でリクエストに付与する。

サーバー側 — Bearer Token 認証

```
import grpc_auth "github.com/grpc-ecosystem/go-grpc-middleware/auth"

func authorize(ctx context.Context) (context.Context, error) {
    token, err := grpc_auth.AuthFromMD(ctx, "Bearer") // メタデータから Bearer トークンを取得
    if err != nil { return nil, err }

    if token != "test-token" {
        return nil, status.Error(codes.Unauthenticated, "token is invalid")
    }
    return ctx, nil
}
```

クライアント側 — メタデータにトークンを付与

```
func callListFiles(client pb.FileServiceClient) {
    md := metadata.New(map[string]string{"authorization": "Bearer test-token"})
    ctx := metadata.NewOutgoingContext(context.Background(), md)

    res, _ := client.ListFiles(ctx, &pb.ListFilesRequest{})
    fmt.Println(res.GetFileNames())
}
```

発展: TLS (暗号化通信) — 証明書の準備 (mkcert)

mkcert を使うと、localhost 用の信頼される自己署名証明書を数コマンドで生成できる。

```
# 1. mkcert 本体をインストール
brew install mkcert

# 2. ローカル CA をシステムにインストール (初回のみ)
mkcert -install

# 3. サーバー用証明書を生成 → server/ssl/ へ出力
cd grpc-lesson/server
mkcert -cert-file ssl/localhost.pem \
      -key-file  ssl/localhost-key.pem \
      localhost

# 4. クライアント検証用に rootCA をコピー
mkdir -p ../client/ssl
cp "$(mkcert -CAROOT)/rootCA.pem" ../client/ssl/rootCA.pem
```

生成物:

- `server/ssl/localhost.pem` / `localhost-key.pem` — サーバー証明書と秘密鍵
- `client/ssl/rootCA.pem` — クライアントが検証に使うルート CA

発展: TLS (暗号化通信) — コード実装

サーバー側

```
// SSL 証明書の読み込み
creds, err := credentials.NewServerTLSFromFile(
    "ssl/localhost.pem",
    "ssl/localhost-key.pem",
)

// TLS 付きで gRPC サーバーを作成
s := grpc.NewServer(grpc.Creds(creds))
```

クライアント側

```
// ルート証明書を読み込み
creds, err := credentials.NewClientTLSFromFile("ssl/rootCA.pem", "")

// TLS 付きで gRPC 接続
conn, err := grpc.NewClient(
    "localhost:50051",
    grpc.WithTransportCredentials(creds),
)
```

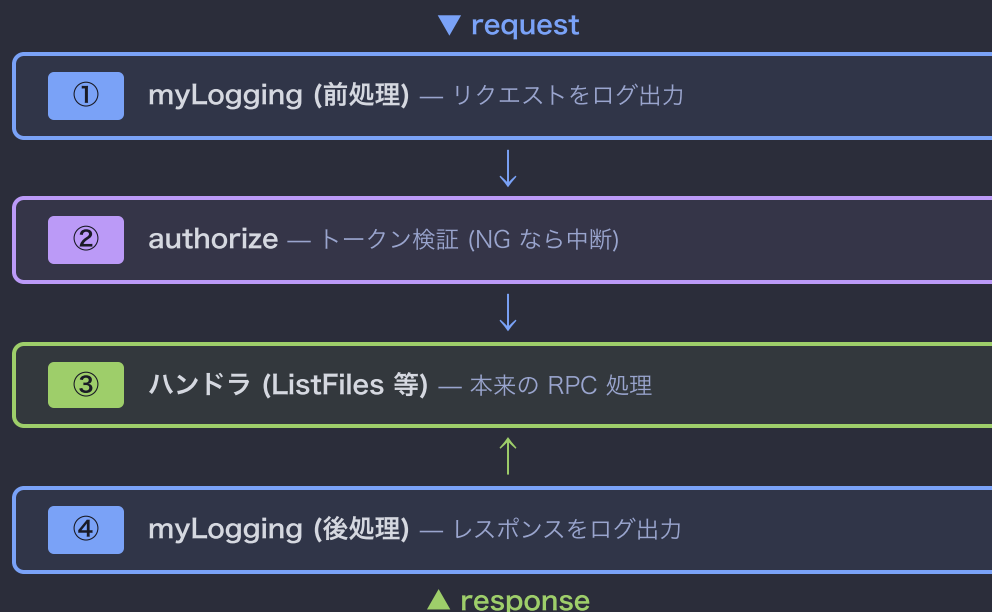

発展: Interceptor のチェーン

複数の Interceptor を組み合わせる

```
import grpc_middleware "github.com/grpc-ecosystem/go-grpc-middleware"

s := grpc.NewServer(
    grpc.Creds(creds),
    grpc.UnaryInterceptor(
        grpc_middleware.ChainUnaryServer(
            myLogging(), // 1. ログ出力
            grpc_auth.UnaryServerInterceptor(authorize), // 2. 認証
        ),
    ),
)
pb.RegisterFileServiceServer(s, &server{})
```

処理の流れ: Interceptor は登録順に**玉ねぎ状**にハンドラを包む。リクエストは外側 (①) から内側 (③) へ通過し、最深部の本処理を経たレスポンスが逆順 (③ → ④) で外側へ戻る。各 Interceptor 内の `handler(ctx, req)` の**呼び出し前後**にコードを書くことで、同じ Interceptor で**前処理と後処理**の両方を扱える。



エラーハンドリング

gRPC ステータスコード

```
import ( "google.golang.org/grpc/codes"; "google.golang.org/grpc/status" )

// サーバー側: エラーを返す
return status.Errorf(codes.NotFound, "file not found: %s", filename)
return status.Error(codes.Unauthenticated, "token is invalid")
```

主要なステータスコード

コード	説明	HTTP 相当
OK	成功	200
NotFound	リソースが見つからない	404
Unauthenticated	認証エラー	401
PermissionDenied	権限なし	403
DeadlineExceeded	タイムアウト	504
InvalidArgument	不正な引数	400
Internal	内部エラー	500

動作確認

実行手順

```
# ターミナル 1: サーバーを起動
cd grpc-lesson/server
go run main.go
# → Server is running on port 50051

# ターミナル 2: クライアントを実行
cd grpc-lesson/client
go run main.go
```

注: TLS 証明書 (`ssl/localhost.pem`) とストレージ (`../storage/`) を相対パスで参照しているため、サーバー / クライアントの各ディレクトリから起動すること。

確認ポイント

- **Unary:** `ListFiles` でファイル一覧が取得できるか
- **Server Streaming:** `Download` でデータが5バイトずつ受信されるか
- **Client Streaming:** `Upload` でファイルサイズが返ってくるか
- **Bidirectional:** `UploadAndNotifyProgress` で進捗メッセージが表示されるか

まとめ

Protocol Buffers

- `.proto` ファイルでスキーマを定義し、`protoc` でコードを自動生成
- バイナリフォーマットで JSON より高速・軽量
- 型安全で、言語を跨いだ通信に最適

gRPC

- HTTP/2 ベースの高性能 RPC フレームワーク
- **4つの通信パターン**で多様なユースケースに対応
- Interceptor で横断的関心事（ログ・認証）を分離
- TLS による暗号化通信をサポート

参考教材

- Go言語で学ぶ実践gRPC入門 (Udemy)
- gRPC 公式ドキュメント / Protocol Buffers 公式ドキュメント